Tank Shield - Wiring Guide and Configuration

Firmware: `Embedded_Project_Tank_2025_2026`
Tank: Arduino Uno R4 WiFi + Emakefun PS2X & Motor Drive Board
Controller: ESP32 Dev Module
Updated: July 19, 2026

1. General map

Function	Connection	Notes
Left track	Shield `M3`	2-wire DC motor
Right track	Shield `M1`	2-wire DC motor
Unused port	Shield `M2`	Left unused because on tested hardware it rotated by itself
Horizontal turret	`D2-D5` to driver `IN1-IN4`	5-wire stepper with external driver
Elevation servo A	Shield `S5`	Logical angle direct `0-47` degrees
Elevation servo B	Shield `S6`	Mirrored motion: `90 - angle`
Cannon relay	`D7` to relay `S` pin	Power terminals `COM` and `NO`
Communication	WiFi AP + UDP	`Tank_AP`, password `12345678`, port `4210`

Motors and servos are driven by the shield. The horizontal turret and the relay use Arduino digital pins routed via the shield connectors.

2. DC tracks

Each yellow motor has two wires and must use both terminals of the same motor port. Do not connect one motor wire to GND.

Track	Port	Connection
Left	`M3`	Two motor wires into the two terminals of `M3`
Right	`M1`	Two motor wires into the two terminals of `M1`

If a track spins in the wrong direction, swap its two wires or change in `tank/src/trackController.h`:

```
#define LEFT_TRACK_INVERTED false
#define RIGHT_TRACK_INVERTED false
```

Set `true` only for the side that needs to reverse direction.

3. Horizontal turret

The 5-wire stepper is not connected directly to the shield: its external driver powers the coils. Arduino only sends the digital sequence on the four inputs.

Arduino/shield	Turret driver
D2	A / IN1
D3	B / IN2
D4	C / IN3
D5	D / IN4
GND	GND driver

Power the driver with the voltage required by its module. Its GND must be connected to the common GND of Arduino, shield, ESP32 and any power supplies connected via cable and the buck converter.

The firmware uses a single-phase full-step sequence:

```
A -> B -> C -> D -> A       rotation one direction
D -> C -> B -> A -> D       rotation the other direction
```

Speed depends on `TURRET_STEP_INTERVAL_MS` in `tank/src/servoTorreta.h`: higher values slow the turret. Current value is `2 ms`.

4. Elevation servos

The servo connector on the shield has three pins: `G` = ground, `V` = supply, `S` = signal. Always check the letters printed on the board before inserting the connector.

Servo	Port	Current command
Servo A	S5	from 0 to 47 degrees
Servo B	S6	from 90 to 43 degrees, i.e. `90 - angle A`

The value shown on the tank serial is the logical angle of servo A. Servo B automatically receives the mirrored command.

Configuration in `tank/src/servoTorreta.cpp`:

```
#define ELEVATION_MIN_ANGLE 0
#define ELEVATION_MAX_ANGLE 47
#define ELEVATION_MIRROR_BASE 90
```

5. Relay and cannon

Control side

Relay	Arduino/shield
+	5V
-	GND
S	D7

Power side

Use COM and NO, not NC:

```
cannon positive supply -> COM relay
NO relay                    -> cannon positive input
negative supply             -> cannon negative input
```

Looking at the module as in the project photo, the upper terminal toward the cannon is NO. Always verify the NO/COM/NC labeling on the PCB before powering.

The relay receives a 200 ms pulse. At least 12 seconds must elapse between two firings, even if the button is pressed multiple times.

6. ESP32 controller

Physical connections

Component	Physical input	ESP32 pin
Joy1 drive	X axis	GPIO 34
Joy1 drive	Y axis	GPIO 32
Joy2 turret	X axis	GPIO 35
Joy2 turret	Y axis	GPIO 33
Zero button	signal	GPIO 25, button to GND
Fire button	signal	GPIO 26, button to GND

Buttons use INPUT_PULLUP: released = HIGH, pressed = LOW.

Swap X/Y in software

Joysticks are mounted rotated. The firmware swaps X and Y on both:

```
#define DRIVE_SWAP_X_Y true
#define TURRET_SWAP_X_Y true
```

After the swap, logical commands sent over UDP are:

Logical command	Read physical pin	Function
`driveX`	GPIO 32	differential steering
`driveY`	GPIO 34	forward/backward
`turretX`	GPIO 33	horizontal turret
`elevationY`	GPIO 35	elevation servo

To invert only the direction of an axis use the four constants `DRIVE_X_INVERTED`, `DRIVE_Y_INVERTED`, `TURRET_X_INVERTED` and `ELEVATION_Y_INVERTED` in `controller-esp32/src/joystickReader.cpp`. Joy1 axis orientation is corrected only here; on the tank only the two physical motor inversions remain (`LEFT/RIGHT_TRACK_INVERTED`).

7. Joystick calibration

On ESP32 startup:

1. Leave both joysticks centered.
2. Firmware waits `600 ms`.
3. Reads each axis `80` times, 2 ms apart, and computes the real center. Full calibration therefore takes about `1.24 s` plus small processing overhead.
4. Calibration is accepted only if each axis is near `512` (maximum deviation `70`) and does not oscillate more than `40` points during sampling.
5. Remaps the measured center to `512`, keeping extremes `0-1023`. If calibration is invalid, only the safe command is sent and a restart with joysticks centered is required.
6. Even with valid calibration, before arming the controls it requires neutral joysticks and released buttons for `400 ms`.

Deadzones are not all equal: each filter serves a different purpose.

Filter	Value and dead zone
Joy1 drive on controller	`DRIVE_INPUT_DEADZONE = 20`: less than 20 points from calibrated center
Joy2 turret/elevation on controller	`TURRET_INPUT_DEADZONE = 80`: less than 80 points from calibrated center
Tracks on the tank	`TRACK_COMMAND_DEADZONE = 100`: less than 100 points from UDP center, before and after mixing
Turret and servos on the tank	`TURRET_JOYSTICK_DEADZONE = 200`: less than 200 points from UDP center

The small Joy1 deadzone avoids adding an excessive dead zone to the tank protection. Do not touch joysticks during initial calibration and during the arming wait.

8. Differential drive and speed

Joy1 Y controls forward/backward linearly and can reach maximum speed. Joy1 X controls steering only. There are no additional curves, bands or intermediate steering thresholds.

```
forward  = decode(driveY)
turn     = decode(driveX)
left     = constrain(forward + turn, -512, +512)
right    = constrain(forward - turn, -512, +512)
```

`decode()` scales the input from `0-1023`, subtracts center `512` and applies the tank deadzone. With joystick Y forward both tracks get the same command; with Joy1 X one side speeds up and the other slows down. With only Joy1 X the two tracks turn in opposite directions and the tank spins in place. `constrain()` prevents the sum from exceeding the allowed command range.

Main configurations in `tank/src/trackController.h`:

```
#define TRACK_COMMAND_DEADZONE 100
#define LEFT_TRACK_MIN_PWM 900
#define RIGHT_TRACK_MIN_PWM 900
```

When a reverse command is requested, each track is first braked to zero PWM and re-enabled after `30 ms` (`TRACK_REVERSE_DEAD_TIME_MS` in `tank/src/trackController.cpp`). The pause applies even if the joystick briefly passes through center before the opposite direction.

9. UDP protocol and rate

The controller sends about `50 packets per second`, one every `20 ms`:

`V1;driveX;driveY;turretX;elevationY;zero;fire`

Example:

`V1;512;900;512;512;0;0`

The tank keeps the last valid command. If it does not receive valid packets for `200 ms` it centers all axes, deactivates buttons and stops movements. On the first timeout it returns the safe command before performing a new `WiFiS3` polling, so it does not intentionally add a network wait to the normal stop.

On startup the tank remains disarmed. If `WiFi.beginAP()` or opening the UDP socket fail, it does not enter `while(true)`: it remains in the safe state and retries every `3 seconds`. When AP and UDP are ready, and after any boot, timeout or network fault, the tank requests `3` consecutive packets with all axes within `+/-20` of center and buttons released before rearming actuators.

If the ESP32 loses `Tank_AP`, it does not block the program: it stops sending, tries to reconnect every `3 seconds` and allows the tank to enter the safety timeout. When WiFi returns it updates the IP from the gateway, reopens UDP

on port 4210, requests controller neutrality again and resumes sending only after rearming.

The ESP32 error endPacket(): could not send data: 12 is ENOMEM: a WiFi stack buffer is temporarily exhausted, it is not a V1 string overflow. An isolated error leaves the socket open; after 3 consecutive failures the controller closes the socket and tries to reopen it after 200 ms without forcing an immediate WiFi reconnection. Radio sleep is disabled to reduce latency. If it appears together with WiFi lost, check power and motor electrical noise first: software cannot eliminate brown-out or EMI.

10. Power

According to the Emakefun V1.4 manual:

- the shield can be powered from the Arduino DC jack with 6-12 V;
- a 7.4 V battery on the Arduino jack is therefore within the expected range;
- with the servo jumper at 5V, servos use the shield's 5 V regulator;
- with the jumper at EX, servos use the external power input;
- the manual specifies up to 3 A for the shield's 5 V rail, but the actual consumption of the two servos depends on load and stall peaks.

Recommended configuration when servos jerk or reset Arduino:

```
battery 7.4 V -> Arduino DC jack
battery 7.4 V -> buck converter input
buck regulated output -> shield external servo power
servo jumper -> EX
GND buck -> GND Arduino/shield
```

Adjust and measure the buck output before connecting servos. Use the voltage allowed by the servos, typically 5-6 V. Do not connect two different supplies to the same positive rail and do not power motors or servos from USB alone.

For WiFi problems during driving, keep logic/controller power separate from the motor branch when possible, with a correct common ground. Add a suitable electrolytic nearby the shield (approx. 470–1000 uF, correct voltage and polarity), bypass ceramics and noise-suppression capacitors at motor terminals; twist motor wires and keep them away from the ESP32 antenna. Before adding components, verify current and voltage limits of battery, shield and motors.

> Important: a software timeout is not an electrical switch. To guarantee a stop even if WiFiS3/I2C block or the Uno resets, provide a hardware enable/OE/ STBY default-off, or an E-stop that physically removes motor and relay enable.

11. Where to change behavior

Change	File	Constant/function
Swap X/Y Joy1	controller-esp32/src/joystick_reader.cpp	DRIVE_SWAP_XY
Swap X/Y Joy2	controller-esp32/src/joystick_reader.cpp	TURRET_SWAP_XY
Invert an axis	controller-esp32/src/joystick_reader.cpp	DRIVE_INVERTED
Joy1 deadzone (drive)	controller-esp32/src/joystick_reader.cpp	DRIVE_DEADZONE
Joy2 deadzone	controller-esp32/src/joystick_reader.cpp	TURRET_DEADZONE
Track deadzone	tank/src/trackController.h	TRACK_COMMAND_DEADZONE
Single track speed	tank/src/trackController.h	LEFT/RIGHT_TRACK_MIN/MAX_PWM, LEFT/RIGHT_TRACK_PWM_PERCENT
Pause before reversing	tank/src/trackController.cpp	TRACK_REVERSE_DEAD_TIME_MS
Track direction	tank/src/trackController.h	LEFT/RIGHT_TRACK_INVERTED
Turret speed	tank/src/servoTorreta.h	TURRET_STEP_INTERVAL_MS
Elevation limits	tank/src/servoTorreta.cpp	ELEVATION_MIN/MAX_ANGLE
Mirror second servo	tank/src/servoTorreta.cpp	ELEVATION_MIRROR_BASE
Relay pulse duration	tank/src/main.cpp	FIRE_RELAY_PULSE_MS
Pause between shots	tank/src/main.cpp	FIRE_RELAY_COOLDOWN_MS
Tank AP/UDP retry	tank/src/udpReceiver.cpp	NETWORK_RETRY_INTERVAL_MS
Tank UDP rearm	tank/src/udpReceiver.cpp	REARM_NEUTRAL_PACKET_COUNT, REARM_NEUTRAL_TOLERANCE
UDP timeout	tank/src/udpReceiver.cpp	CONNECTION_TIMEOUT_MS
Controller send rate	controller-esp32/src/main.cpp	UDP_SEND_INTERVAL_MS
ESP32 ENOMEM recovery	controller-esp32/src/udp_sender.cpp	UDP_SEND_FAILURE_LIMIT, UDP_SOCKET_RECOVERY_INTERVAL_MS
Track I2C refresh	tank/src/trackController.h	TRACK_COMMAND_REFRESH_INTERVAL_MS

12. Quick diagram

```
ESP32 controller
  GPIO34 <- Joy1 physical X -> driveY after swap
  GPIO32 <- Joy1 physical Y -> driveX after swap
  GPIO35 <- Joy2 physical X -> elevationY after swap
  GPIO33 <- Joy2 physical Y -> turretX after swap
  GPIO25 <- zero button to GND
  GPIO26 <- fire button to GND
                 |
                 | WiFi UDP, every 20 ms
                 v
Arduino Uno R4 WiFi + shield
  M3     -> left track DC motor, both wires
  M1     -> right track DC motor, both wires
  D2-D5  -> IN1-IN4 turret stepper driver
  S5     -> elevation servo A
  S6     -> elevation servo B
  D7     -> S relay cannon
  Relay  -> COM + NO, NO toward cannon
```

13. Pre-power checks

- No motor wire connected directly to GND.
- Turret driver and Arduino share common GND.
- Servo connectors oriented according to `G/V/S` printed on the shield.
- Measure the buck converter with a multimeter before connecting servos.
- Servo jumper set consistently: `5V` or `EX`.
- Relay connected to `COM` and `NO`, not to `NC`.
- Joysticks centered during ESP32 startup.
- Tracks lifted for the first test of timeout or `ENOMEM` recovery.
- Measure controller supply while tracks start/reverse; if `WiFi lost` appears, fix the power branch and wiring first.